Funcionament i Pinout de l'LCD1602

L'LCD1602 és una pantalla de cristall líquid de dues línies amb 16 caràcters per línia. Està construïda sobre un controlador **HD44780**, que és un estàndard per a aquest tipus de pantalles. Aquest controlador gestiona la visualització dels caràcters i la seva col·locació en el panell, a més de permetre enviar ordres i dades de manera controlada.

La pantalla **LCD1602** s'utilitza en mode **4 bits** o **8 bits** per la comunicació de dades:
- **Mode 8 bits**: envia 8 bits de dades en cada instrucció (més ràpid però consumeix més pins).
- Mode 4 bits: només s'envien 4 bits per instrucció, primer els 4 bits alts i després els 4 bits baixos (consumeix menys pins però és lleugerament més lent).

Pinout de l'LCD1602

A continuació tens la llista dels pins i la seva funció en detall:

1. **VSS (Pin 1):** Terra (GND). Connecta aquest pin a GND del sistema.

2. **VDD (Pin 2): A**limentació positiva. Sol ser de 5V.

3. **VO (Pin 3):** Control de contrast de la pantalla. **A**quest pin s'utilitza per ajustar el contrast amb un potenciometre. Generalment, es connecta un extrem del potenciometre a GND i l'altre a 5V, amb el pin central connectat al VO.

4. **RS** (Pin 4): Selector de registre. Indica si el pin d'entrada es una instrucció o dades:

- **RS** = 0: Instrucció.

- **RS** = 1: Dades.

5. **RW** (Pin 5): Llegeix o escriu dades:

- **RW** = 0: **E**scrivim a la pantalla.

- **RW** = 1: Llegim de la pantalla (poques vegades s'utilitza aquest mode; normalment es deixa connectat a GND).

6. **E** (Pin 6): **E**nable (**A**ctivador). **A**quest pin activa la transferència de dades. Una transició de baix a alt indica que es llegeixen les dades en els pins de dades.

7. **D0-D7** (Pins 7 a 14): Pins de dades. **E**s poden utilitzar els 8 pins per a mode de 8 bits o nomes D4-D7 per a mode de 4 bits.

8. **A** (Pin 15): **A**limentació per al retroil·luminat. Connecta'l a 5V per il·luminar la pantalla.

9. **K** (Pin 16): Terra del retroil·luminat. Connecta'l a GND.

Exemple de Funcionament a Baix Nivell

- **Instruccions de Configuració**: Quan configures la pantalla, l'envies al mode de 4 o 8 bits amb una instrucció específica. Això estableix com es comunicarà amb la Raspberry Pi o un altre microcontrolador.

- **Enviar una Instrucció o Dada**: Primer estableixes el valor de **RS** segons si és una instrucció o dada, després el valor de **RW** (normalment 0 per escriure) i actives el pin **E** per indicar que la pantalla ha de processar l'entrada.

Bienvenido.

Gracias por elegir nuestra pantalla LCD de AZ-Delivery. Estos están disponibles individualmente o en paquete con convertidor I2C en verde o azul, y en dos tamaños: 16x02 y 20x04. En las siguientes páginas le explicaremos cómo configurar y utilizar el disposit









Introducción

Una pantalla de cristal líquido, o LCD, es un dispositivo que utiliza cristales líquidos para mostrar caracteres visuales.

Los cristales líquidos no emiten luz por sí mismos. Las pantallas LCD utilizan una retroiluminación y cristales líquidos para bloquear la luz. Cuando no circula corriente por los cristales líquidos, éstos se encuentran en un estado caótico. La luz de la retroiluminación atraviesa los cristales líquidos sin esfuerzo. Cuando la corriente fluye a través de los cristales líquidos, éstos se organizan en un estado uniforme. Esto crea una sombra en la pantalla que crea objetos.

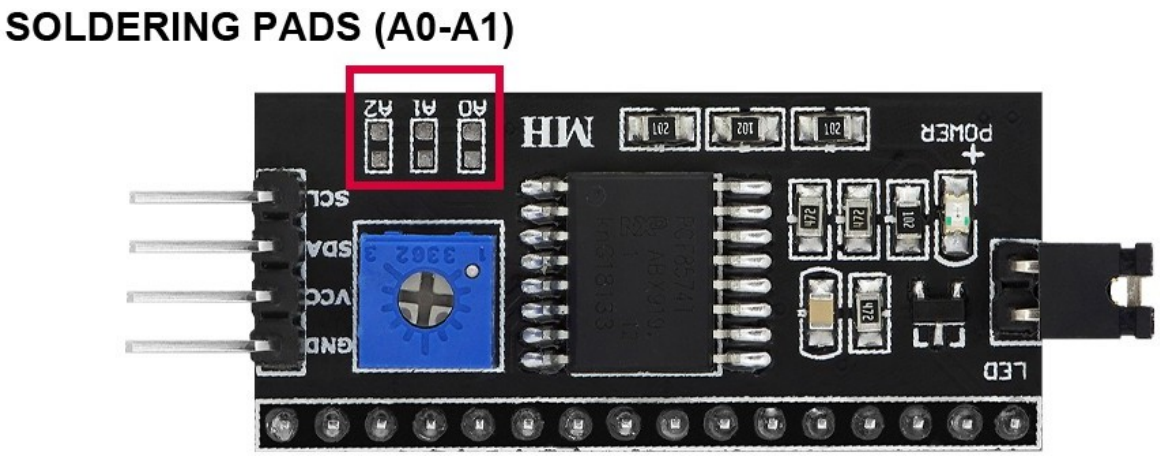
En pocas palabras, los LCD utilizan cristales líquidos para pasar o bloquear la luz procedente de la retroiluminación. Así se crean los píxeles. Se combinan para mostrar cualquier objeto 2D visible en la pantalla. Cada área de píxeles puede encenderse y apagarse suministrando electricidad a través de un chip controlador integrado.

Cada segmento de la pantalla de cristal líquido representa un píxel y debe controlarse por separado. Por esta razón, se requiere un circuito integrado especial llamado chip controlador. El LCD 16x02(20x04) tiene un chip controlador llamado "HD44780". Para controlar la pantalla, el microcontrolador debe comunicarse con el chip controlador. El chip controlador utiliza un tipo de interfaz SPI para comunicarse con el microcontrolador.

El adaptador I2C

El adaptador I2C es un dispositivo que simplifica la conexión entre las pantallas LCD y los microcontroladores. Utiliza la interfaz I2C para comunicarse con el microcontrolador. Se pueden conectar varios adaptadores a la misma interfaz I2C. El adaptador es compatible con pantallas LCD, con chips HD44780 integrados. El adaptador I2C es compatible tanto con paneles LCD 16x02 como con paneles LCD 20x04 ofrecidos por AZ-Delivery.

El adaptador I2C viene con una dirección I2C predefinida, que es 0x27. Sin embargo, se puede cambiar soldando los pads etiquetados A0, A1 y A2 en el adaptador.



En la siguiente t a b l a se muestra cómo configurar una dirección I2C específica del adaptador:

PADS			DIRECCIÓN I2C
A2	**A1**	**A0**	
C	C	C	0x20
C	C	O	0x21
C	O	C	0x22
C	O	O	0x23
O	C	C	0x24
O	C	C	0x25
O	O	C	0x26
O	O	O	0x27
O - Abierto			**C - Cerrado**

Datos técnicos

" Tensión de funcionamiento: de	3,3 V a 5 V
" Área de visualización:	12 x 56 mm
" Tipo LCD:	STN,positivo,transflectivo,verde/azul
" Retroiluminación:	ED, blanco
" Punto de vista:	180°
" Modos:	paralelo (8 bits y 4 bits)
" Temperatura de funcionamiento:	-10 °C a 60 °C
"Dimensiones:	36x 80 x 12,5 mm [1,4 x 3,1 x 0,5 pulgadas].
" Interfaz:	I2C/paralelo
" Dirección I2C:	0x20 - 0x27
" Ajuste del contraste :	Potenciómetro
" Ajuste del fondo.:	Puente

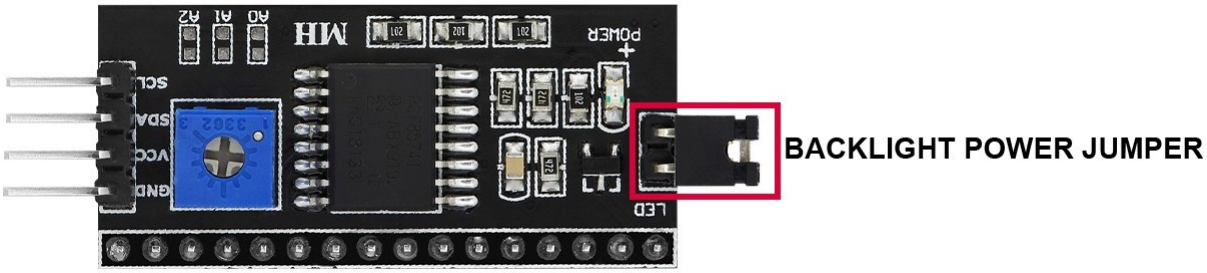
La tensión de la retroiluminación es de 5 V CC. Para las funciones lógicas el consumo de corriente en funcionamiento es de 1,5mA y para la retroiluminación de 30mA.

La resolución de la pantalla es 16x02, lo que significa que los caracteres pueden mostrarse en 2 líneas con 16 caracteres por línea. Cada carácter consta de 5x7 píxeles.

Para ajustar el contraste de la pantalla, el adaptador I2C tiene un potenciómetro incorporado. Por lo tanto, se necesita un pequeño

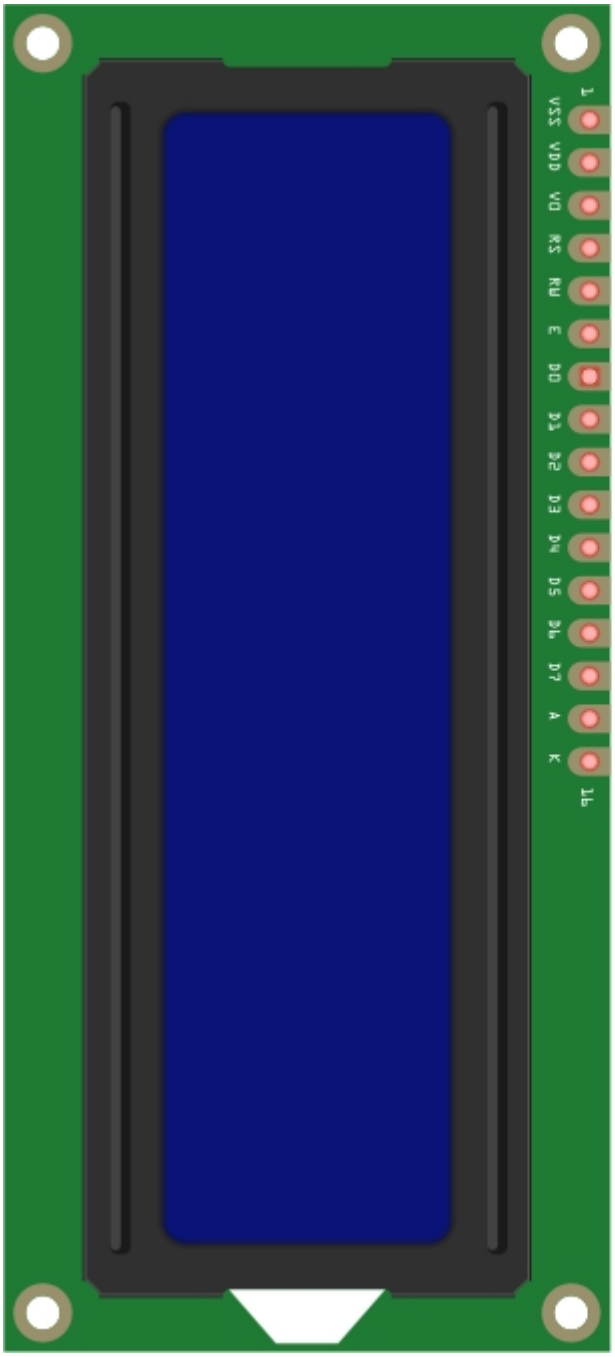
destornillador para el ajuste.

Para controlar la retroiluminación de la pantalla LCD, el adaptador I2C tiene un puente de alimentación para la retroiluminación. El puente se utiliza para encender/apagar la luz de fondo. Cuando el puente está conectado, se utiliza para conectar la fuente de alimentación para la luz de fondo. Cuando el jumper está desconectado, la fuente de alimentación de la retroiluminación está desconectada.



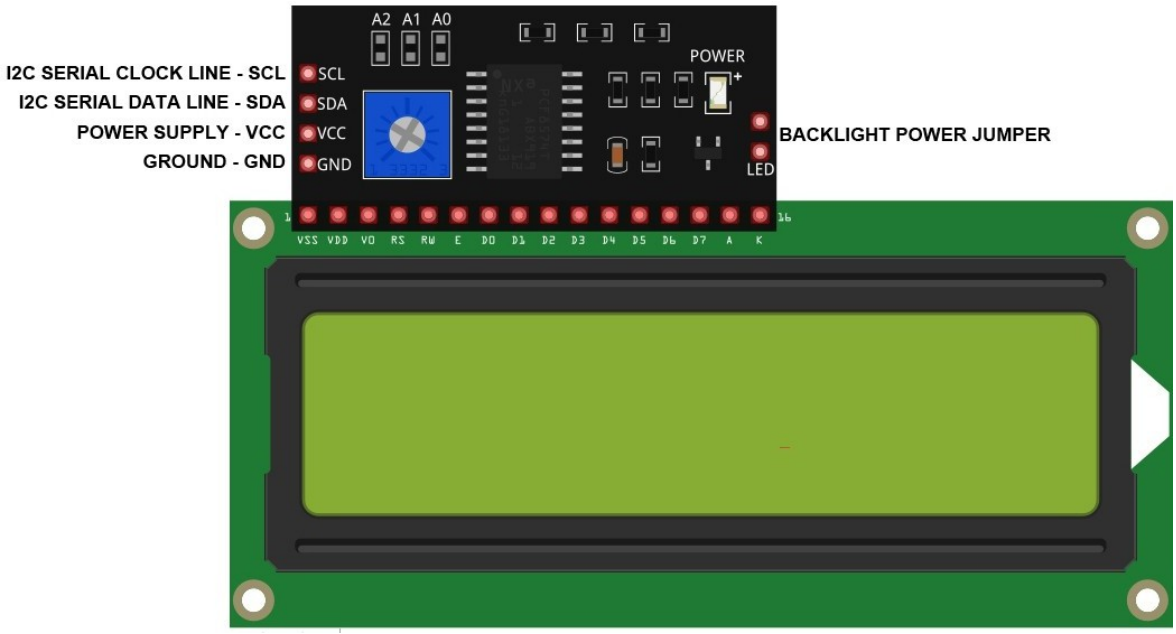
Pinout sin adaptador I2C

La pantalla LCD 16x02 tiene 16 pines. La asignación de pines es la siguiente (también se aplica al 20x04):



Pinout con adaptador I2C

El LCD 16x02 tiene 16 pines y el adaptador I2C tiene 20. El adaptador se conecta al LCD de la siguiente manera (esto también se aplica al 20x04):

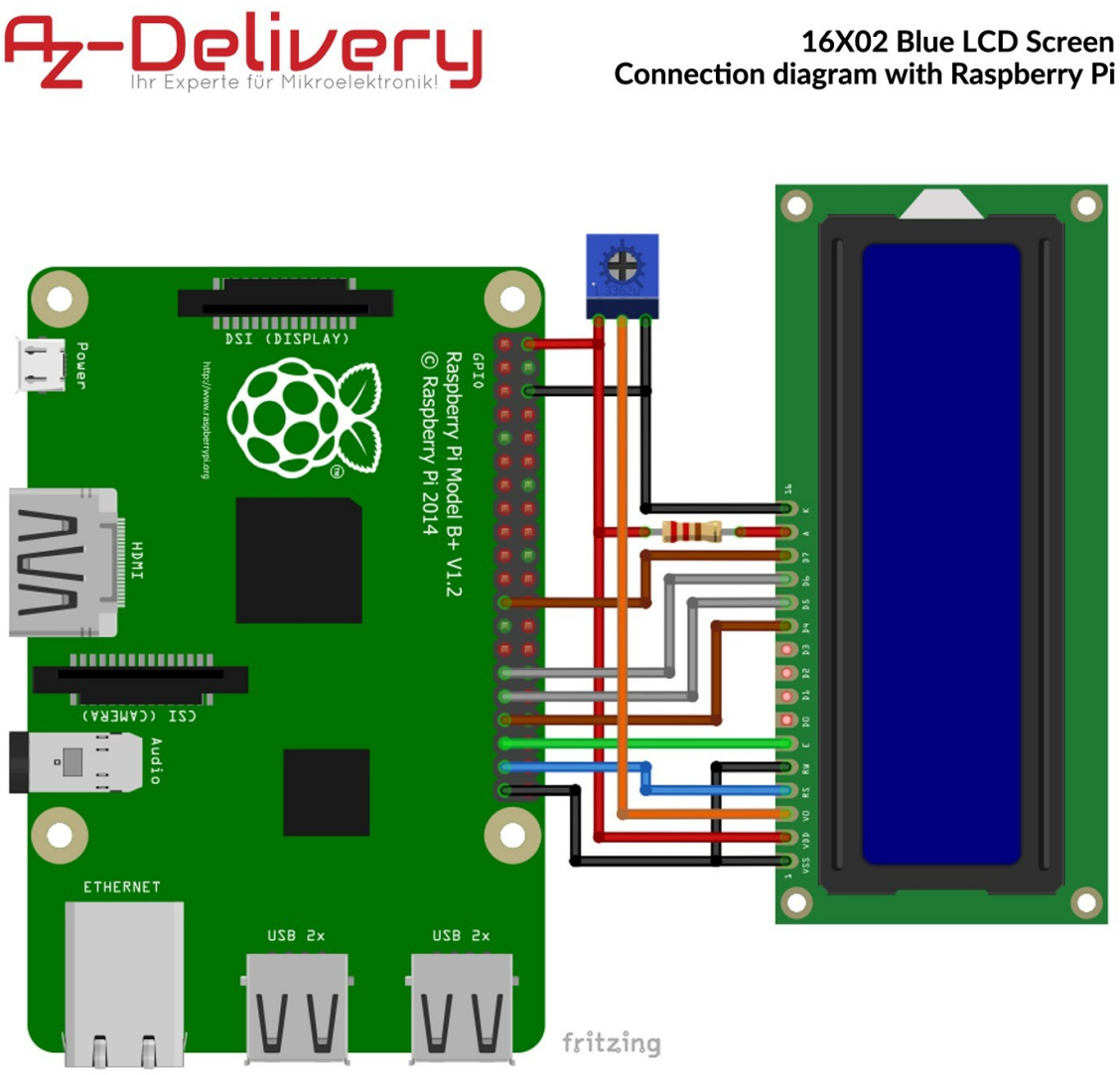


Nota: Es necesario conectar el adaptador I2C y la pantalla LCD exactamente como se muestra arriba. Si se conectan de forma diferente, los dispositivos pueden dañarse.

Nota para la Raspberry Pi: El voltaje del nivel lógico TTL de los pines de E/S es de 5V. Para utilizar el panel LCD y el adaptador I2C con la Raspberry Pi, es necesario utilizar un conversor de nivel lógico. De lo contrario, la alimentación de la señal a través de los pines de E/S del módulo a los pines GPIO de la Raspberry Pi puede causar daños. Por lo tanto, utilice el TXS0108E 8ch Logic Level Converter. ofrecido por AZ-Delivery.

Conexión de la pantalla a la Raspberry Pi

Conecte el módulo a la Raspberry Pi como se muestra a continuación (se aplica también al 20x04):



Pantalla Pin	Pin de Raspberry Pi	Pin físico	color del cable
VSS	GND	6	**cable negro**
VDD	5V	2	**cable rojo**
RS	GPIO26	37	**cable azul**
RW	GND	39	**cable negro**
E	GPIO19	35	**cable verde**
D4	GPIO13	33	**cable marrón**
D5	GPIO6	31	**cable gris**
D6	GPIO5	29	**cable gris**
D7	GPIO11	23	**cable marrón**
K	GND	6	**cable rojo**
A	5V, mediante resistencia de 220Ω	2	**cable naranja**
V0	clavija central del potenciómetro		

	Potenciómetro		
GND	derecha Pin	6	**cable negro**
5V	izquierda Pin	2	**cable rojo**

Python- Skript

Se crean dos scripts, uno para todas las funciones y otro para utilizar estas funciones. A continuación encontrará el código para el primer script:

```
import RPi.GPIO as GPIO
import tiempo

LCD_RS = 26
LCD_E  = 19
LCD_D4 = 13
LCD_D5 = 6
LCD_D6 = 5
LCD_D7 = 11

# Define algunas constantes del dispositivo
LCD_WIDTH = 16#   Caracteres máximos por línea, cambiar a "20 "si
20x04
LCD_CHR = Verdadero
LCD_CMD = Falso

LCD_LINE_1 = 0x80 # Dirección LCD RAM para la 1ª línea
LCD_LINE_2 = 0xC0 # Dirección LCD RAM para la 2ª línea

# Constantes de
tiempo E_PULSE =
0.0005
E_RETRASO = 0,0005
```

```
  # una
  pestaña
def lcd_init(RS, E, D4, D5, D6, D7):
  GPIO.setwarnings(False)
  GPIO.setmode(GPIO.BCM)
  global LCD_RS, LCD_E, LCD_D4, LCD_D5, LCD_D6, LCD_D7
  LCD_RS = RS
  LCD_E = E
  LCD_D4 = D4
  LCD_D5 = D5
  LCD_D6 = D6
  LCD_D7 = D7
  GPIO.setup(LCD_E, GPIO.OUT) # E
  GPIO.setup(LCD_RS, GPIO.OUT) # RS
  GPIO.setup(LCD_D4, GPIO.OUT) # DB4
  GPIO.setup(LCD_D5, GPIO.OUT) # DB5
  GPIO.setup(LCD_D6, GPIO.OUT) # DB6
  GPIO.setup(LCD_D7, GPIO.OUT) # DB7
  # Inicializar pantalla
  lcd_byte(0x33, LCD_CMD) # 110011 Inicializar
  # 110010 Inicializar
  lcd_byte(0x32, LCD_CMD)
  # 000110 Dirección de movimiento del cursor
  lcd_byte(0x06, LCD_CMD)
  # 001100 Pantalla Encendida,Cursor Apagado,
  Parpadeo Apagado lcd_byte(0x0C, LCD_CMD)
  # 101000 Longitud de datos, número de líneas, tamaño de
  fuente lcd_byte(0x28, LCD_CMD)
  # 000001 Borrar pantalla
  lcd_byte(0x01, LCD_CMD)
  time.sleep(E_DELAY)
```

```
  # una
  pestaña
def lcd_byte(bits, modo):
  GPIO.output(LCD_RS, mode) # RS
  GPIO.output(LCD_D4, False)
  GPIO.output(LCD_D5, False)
  GPIO.output(LCD_D6, False)
  GPIO.output(LCD_D7, False)
  if bits & 0x10 == 0x10:
    GPIO.output(LCD_D4, True)
  if bits & 0x20 == 0x20:
    GPIO.output(LCD_D5, True)
  if bits & 0x40 == 0x40:
    GPIO.output(LCD_D6, True)
  si bits & 0x80 == 0x80:
    GPIO.output(LCD_D7, True)
  lcd_toggle_enable() # Alternar pin 'Enable'
  GPIO.output(LCD_D4, False)
  GPIO.output(LCD_D5, False)
  GPIO.output(LCD_D6, False)
  GPIO.output(LCD_D7, False)
  if bits & 0x01 == 0x01:
    GPIO.output(LCD_D4, True)
  if bits & 0x02 == 0x02:
    GPIO.output(LCD_D5, True)
  if bits & 0x04 == 0x04:
    GPIO.output(LCD_D6, True)
  if bits & 0x08 == 0x08:
    GPIO.output(LCD_D7, True)
  # Alternar pin 'Enable'
  lcd_toggle_enable()
```

```
  # una
  pestaña
def lcd_toggle_enable():
  # Activar
  time.sleep(E_DELAY)
  GPIO.output(LCD_E, True)
  time.sleep(E_PULSE)
  GPIO.output(LCD_E, False)
  time.sleep(E_DELAY)

def lcd_string(mensaje, línea):
  # Enviar cadena a la
  pantalla LCD_LINE_1 = 0x80
  LCD_LINE_2 = 0xC0
  message = message.ljust(LCD_WIDTH, " ")
  if line == 0:
    lcd_byte(LCD_LINE_1, LCD_CMD)
  elif line == 1:
    lcd_byte(LCD_LINE_2, LCD_CMD)
  else:
    print('¡Esta lcd tiene dos líneas, línea 0 y línea 1!')
  for i in range(LCD_WIDTH):
    lcd_byte(ord(mensaje[i]), LCD_CHR)

def lcd_clear():
  lcd_byte(0x01, LCD_CMD)
```

Guarda el script con el nombre *lcd16x02.py.*

El código de la secuencia de comandos es un código modificado de la secuencia de comandos en el *enlace*:

A continuación encontrará el código del script principal:

```
importar lcd16x02
from tiempo import sueño
LCD_RS = 26
LCD_E  = 19
LCD_D4 = 13
LCD_D5 = 6
LCD_D6 = 5
LCD_D7 = 11
# Inicializar pantalla
lcd16x02.lcd_init(LCD_RS, LCD_E, LCD_D4, LCD_D5, LCD_D6, LCD_D7)
i = 0
print('[¡Pulsa CTRL + C para finalizar el
script!]') try:
  lcd16x02.lcd_string('AZ-Entrega', 0)
  print('AZ-Entrega')
  print('Imprimiendo variable en el LCD...')
  while True:
    lcd16x02.lcd_string('{}'.format(i), 1)
    i+=1
    sleep(0.001) # 1 milisegundo de retraso

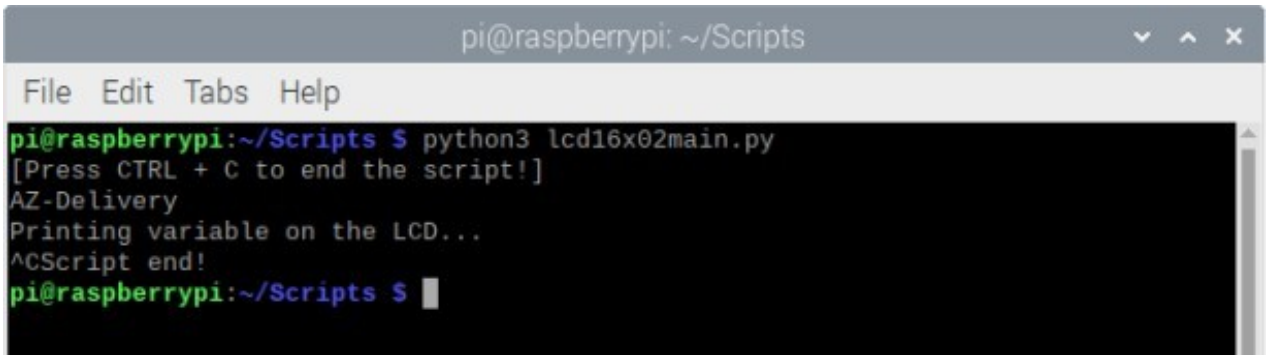
except KeyboardInterrupt:
  print('¡Fin del script!')

finally:
  lcd16x02.lcd_clear()
```

Guarde el script con el nombre lcd16x02main.py en el mismo directorio que el script anterior. Para ejecutar el script, abre el terminal en el directorio donde está guardado el script y ejecuta el siguiente comando:

```
python3 lcd16x02main.py
```

El resultado debería ser el siguiente:



Para detener el script, pulse 'Ctrl + C' en el teclado.

El primer script se utiliza para crear todas las funciones para controlar la pantalla LCD, que no se trata en este eBook. Sólo se explica la función principal del script.

El script principal comienza importando el primer script e importando la función sleep de la librería time.

A continuación, se definen seis variables para representar los pines de la pantalla que están conectados a los pines de la Raspberry Pi.

A continuación, la pantalla se inicializa con la siguiente línea de código: lcd16x02.lcd_init(LCD_RS, LCD_E, LCD_D4, LCD_D5, LCD_D6, LCD_D7).

A continuación se crea la variable "i" y se inicializa con el valor cero. Esto se utiliza para indicar datos cambiantes en la pantalla.

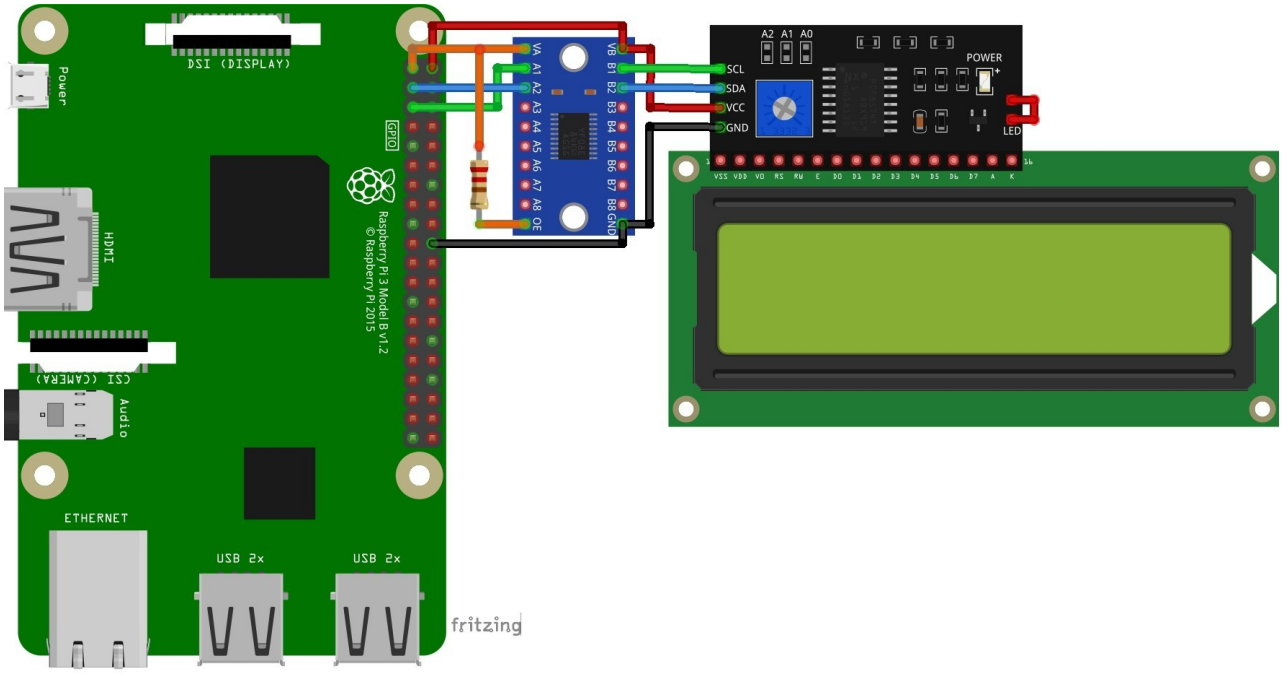
A continuación, se crea un bloque de código try-except-finally. En el bloque try, primero se muestra el mensaje AZ-Delivery en la primera línea de la pantalla y, a continuación, se crea un bloque de bucle indefinido (while True:). En él, la variable i se temporiza en la segunda línea de la pantalla y el valor de la variable i se incrementa en 1. Entre cada repetición del bloque de bucle indefinido hay una pausa de un milisegundo (sleep(0.001)).

El bloque de código except se ejecuta con Ctrl+C. Esto se denomina KeyboardInterrupt Cuando se ha ejecutado este bloque de código, aparece en el terminal el mensaje ¡Fin de script!

El bloque de código finally se ejecuta después del script. Cuando se ha ejecutado el bloque de código finally, se llama a la función lcd_clear(), que borra el búfer de datos de la pantalla.

Conexión de la pantalla a la Raspberry Pi con el adaptador I2C

Conecte la pantalla 16x02 y el adaptador I2C a la Raspberry Pi como se muestra a continuación(se aplica lo mismo a 20x04):



aquí se debe utilizar el convertidor de nivel lógico, porque el adaptador I2C sólo funciona en el rango de 5V. El convertidor de nivel lógico utilizado en este eBook se llama *TXS0108E Logic Level Converter*.

Conecte el adaptador I2C al panel LCD como se muestra en el diagrama de conexión. Asegúrese de que el puente de retroiluminación integrado está conectado (cable rojo, el lado derecho del adaptador I2C en el diagrama de conexión).

Pin adaptador I2C	Pin LLC	Color
SCL	B1	cable verde
SDA	B2	cable azul
VCC	VB	cable rojo
GND	GND	cable negro

Pin LLC	Pin de Raspberry Pi	Clavija física	color
VA	3.3V	1	cable naranja
A1	GPIO3	5	cable verde
A2	GPIO2	3	cable azul
OE	3,3 V (mediante resistencia)	1	cable naranja
GND	GND	20	cable negro
VB	5V	2	cable rojo

Bibliotecas y herramientas para Python

Para utilizar la pantalla con la Raspberry Pi, se recomienda descargar una librería externa. Para descargarla, ve al siguiente *enlace* y descarga el script lcd_class_i2c.py. Guarda el script en el mismo directorio donde está guardado el script del siguiente capítulo.

El script lcd_class_i2c.py utiliza la librería smbus para Python. Si aún no está instalada, abre el terminal y ejecuta los siguientes comandos uno tras otro:

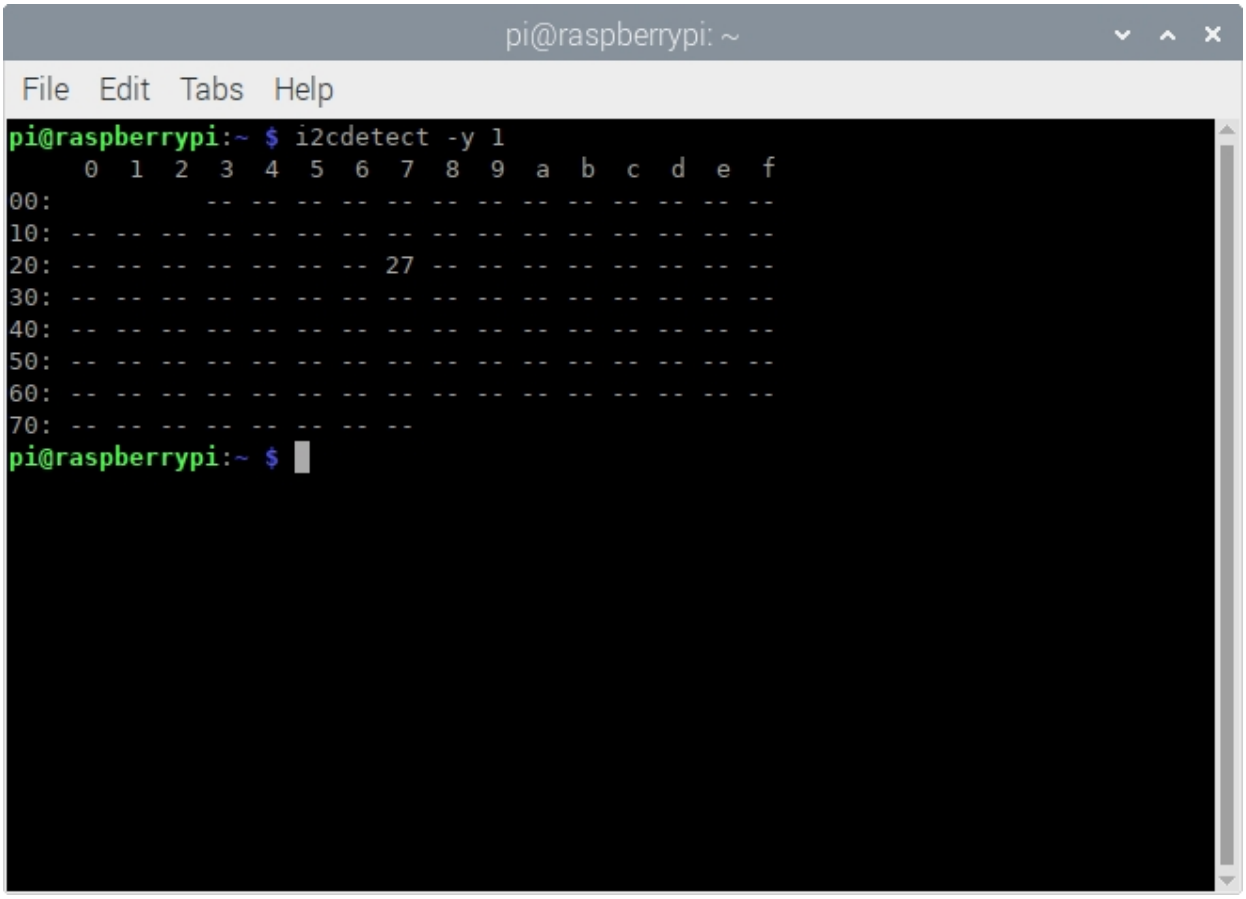
```
sudo apt-get update
sudo apt-get install python3-smbus python3-dev
```

Para determinar la dirección I2C del adaptador I2C, debe estar instalado i2c-tools. Si aún no está instalado, ejecute el siguiente comando en el terminal:

```
sudo apt-get install i2c-tools
```

Para determinar la dirección I2C del adaptador I2C, ejecute el siguiente comando:

```
i2cdetec -y 1
```



donde 0x27 es la dirección I2C del adaptador I2C.

Python-Skript

A continuación encontrará el código del script principal:

```
import lcd_class_i2c as LCD
import time

I2C_ADDR = 0x27
LINE_WIDTH = 16

pantalla = LCD.lcd(line_width=LINE_WIDTH, i2c_address=I2C_ADDR)

print('[¡Pulsa CTRL + C para finalizar el
script!]') try:
  mientras sea verdad:
    print('Imprimiendo mensajes en pantalla')
    screen.lcd_print('AZ-DELIVERY', 'LINE_1', 'CENTER')
    time.sleep(1) # 3 segundos de retraso

    print('Imprimiendo variable en pantalla')
    for i in range(100):
      si i < 10:
        screen.lcd_print('0{}'.format(i), 'LINE_2', 'CENTER')
      else:
        screen.lcd_print('{}'.format(i), 'LINEA_2', 'CENTRO')
      time.sleep(0.00001)
```

```
# una pestaña
    print('Probando
    retroiluminación')
    time.sleep(1)
    screen.backlight('OFF')
    time.sleep(1)
    screen.backlight('ON')
    time.sleep(1)
    print('Borrando la
    pantalla\n') # Pantalla en
    blanco screen.clear_screen()
    time.sleep(1)

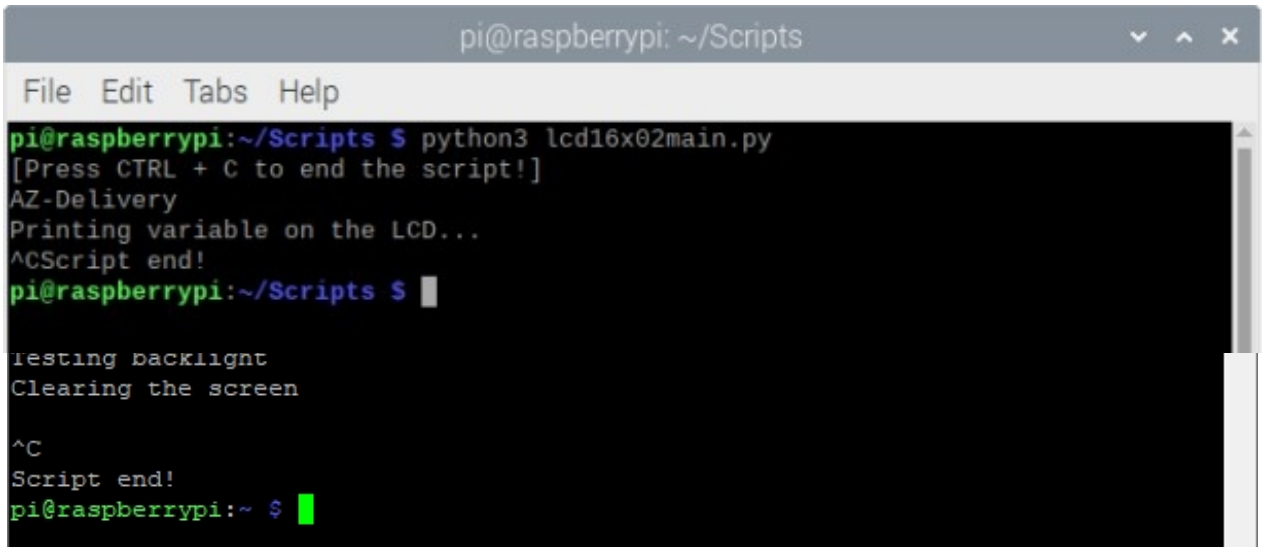
except KeyboardInterrupt:
  print('\n¡Fin del
  script!')

finalmente:
  screen.clear_screen()
```

Guarde el script con el nombre lcd16x02i2c.py en el mismo directorio que el script lcd_class_i2c.py. Para ejecutar el script, abre el terminal en el directorio donde están guardados los scripts y ejecuta el siguiente comando:

`python3 lcd16x02main.py`

El resultado debería ser el siguiente:



Para detener el script, pulse 'Ctrl + C' en el teclado.

La primera secuencia de comandos se utiliza para crear todas las funciones para controlar la pantalla LCD, que no está cubierto en este eBook.

El script principal comienza importando el primer script lcd_class_i2c.py y la librería time.

A continuación, se crea el objeto pantalla. Se utiliza para controlar la pantalla. Creamos el objeto con la siguiente línea de código:

```
screen=LCD.lcd(line_width=LINE_WIDTH, i2c_address=I2C_ADDR)
```

Donde el constructor lcd() tiene dos argumentos. El primer argumento se llama line_width y representa el número de caracteres por línea de la pantalla. El script lcd_class_i2c.py puede usarse tanto para pantallas LCD de 16x02 como para pantallas de 20x04. Así que los valores pasados al argumento line_width son 16 o 20. Cualquier otro valor provocará un error, y el script establece el valor de este argumento en 16. El segundo argumento, llamado i2c_address representa la dirección I2C del adaptador I2C, que en este caso es 0x27.

A continuación se crea un bloque de código try-except-finally. En el bloque try, se crea un bloque de bucle Indefinido (while True:). En el bloque de bucle indefinido, primero se muestra el mensaje AZ-Delivery en la primera línea de la pantalla. El mensaje se sitúa en el centro de la línea. Esto se hace con la siguiente línea de código:

```
screen.lcd_print('AZ-DELIVERY', 'LINE_1', 'CENTER')
```

Donde se utiliza la función lcd_print(). Esta función muestra un mensaje en la pantalla. Tiene tres argumentos y no devuelve ningún valor. El segundo y tercer argumento son opcionales. El primer argumento es una cadena que representa el mensaje mostrado en la pantalla. El segundo argumento, también una cadena, representa la línea en la que se muestra el mensaje. Los valores de este argumento son: LÍNEA_1 o LÍNEA_2. El valor por defecto es LÍNEA_1, que se selecciona si no se utiliza el argumento. Cualquier otro valor provocará un error y el valor se establecerá en el valor por defecto LÍNEA_1. El tercer argumento, también una cadena, representa la alineación del texto en la línea. Los valores de este argumento son: IZQUIERDA, CENTRO o DERECHA. El valor por defecto es IZQUIERDA, que se selecciona si no se utiliza el argumento. Cualquier otro valor provocará un error y el valor también se establecerá en el valor por defecto IZQUIERDA.

Para mostrar una variable en la pantalla, utilice las siguientes líneas de código:

```
mi_var = 10
screen.lcd_print('{}'.format(mi_var))
```

Para controlar la retroiluminación de la pantalla, se utiliza la función backlight(). Esta función tiene un argumento y no devuelve ningún valor. El valor del argumento es una cadena que sólo puede tener dos valores: ON u OFF. Para encender la pantalla, se utiliza la siguiente línea de código:

```
screen.backlight('ON')
```

Para desactivarlo, utilice lo siguiente:

```
screen.backlight('OFF')
```

Para borrar la pantalla (búfer de datos de la pantalla), utilice la función clear_screen(). Esta función no tiene argumentos y no devuelve ningún valor.

El bloque de código except se ejecuta cuando se pulsa CTRL + C en el teclado. Esto se llama KeyboardInterrupt. Cuando se ha ejecutado este bloque, aparece en el terminal el mensaje ¡Fin de script!

El bloque de código finally se ejecuta después del script. Cuando se ha ejecutado el bloque de código finally, se ejecuta la función clear_screen(). Esta función borra el buffer de datos de la pantalla.